

第 6 章-深度神经网络 – 第 6.1 节-固定基函数的局限

《深度学习基础》课程小组

2026 年 4 月 24 日

目录

1	6.1.1 The curse of dimensionality	3
2	6.1.2 High-dimensional spaces	5
3	6.1.3 Data manifolds	6
4	6.1.4 Data-dependent basis functions	8

In recent years, neural networks have emerged as, by far, the most important machine learning technology for practical applications, and we therefore devote a large fraction of this book to studying them.

Previous chapters have already laid many of the foundations we will need.

In particular, we have seen that linear regression models that comprise linear combinations of fixed nonlinear basis functions can be expressed as neural networks having a single layer of weight and bias parameters.

Likewise, classification models based on linear combinations of basis functions can also be viewed as single-layer neural networks.

These allowed us to introduce several important concepts before we embark on a discussion of more complex multilayered networks in this chapter.

Given a sufficient number of suitably chosen basis functions, such linear models can approximate any given nonlinear transformation from inputs to outputs to any desired accuracy and might therefore appear to be sufficient to tackle any practical application.

However, these models have some severe limitations, and so we will begin our discussion of neural networks by exploring these limitations and understanding why it is necessary to use basis functions that are themselves learned from data.

从线性模型改进：基函数也需要从数据中学习而来。

This leads naturally to a discussion of neural networks having more than one layer of learnable parameters.

These are known as **feed-forward networks** or **multilayer perceptrons**.

We will also discuss the benefits of having many such layers of processing, leading to the key concept of **deep neural networks** that now dominate the field of machine learning.

6.1. Limitations of Fixed Basis Functions

Linear basis function models for classification are based on linear combinations of basis functions $\phi_j(\mathbf{x})$ and take the form

$$y(\mathbf{x}, \mathbf{w}) = f\left(\sum_{j=1}^M w_j \phi_j(\mathbf{x}) + w_0\right) \quad (1)$$

where $f(\cdot)$ is a nonlinear output activation function.

Linear models for regression take the same form but with $f(\cdot)$ replaced by the identity.

These models allow for an arbitrary set of nonlinear basis functions $\{\phi_i(\mathbf{x})\}$, and because of the generality of these basis functions, such models can in principle provide a solution to any regression or classification problem.

This is true in a trivial sense in that if one of the basis functions corresponds to the desired input-to-output transformation, then the learnable linear layer simply has to copy the value of this basis function to the output of the model.

More generally, we would expect that a sufficiently large and rich set of basis functions would allow any desired function to be approximated to arbitrary accuracy.

It would seem therefore that such linear models constitute a general purpose framework for solving

problems in machine learning.

Unfortunately, there are some significant shortcomings with linear models, which arise from the assumption that the basis functions $\phi_j(\mathbf{x})$ are fixed and independent of the training data.

To understand these limitations, we start by looking at the behaviour of linear models as the number of input variables is increased.

1 6.1.1 The curse of dimensionality

Consider a simple regression model for a single input variable given by a polynomial of order M in the form

$$y(x, \mathbf{w}) = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M \quad (2)$$

and let us see what happens if we increase the number of inputs.

If we have D input variables $\{x_1, \dots, x_D\}$, then a general polynomial with coefficients up to order 3 would take the form

$$y(\mathbf{x}, \mathbf{w}) = w_0 + \sum_{i=1}^D w_i x_i + \sum_{i=1}^D \sum_{j=1}^D w_{ij} x_i x_j + \sum_{i=1}^D \sum_{j=1}^D \sum_{k=1}^D w_{ijk} x_i x_j x_k. \quad (3)$$

As D increases, the growth in the number of independent coefficients is $\mathcal{O}(D^3)$, whereas for a polynomial of order M , the growth in the number of coefficients is $\mathcal{O}(D^M)$ (Bishop, 2006).

We see that in spaces of higher dimensionality, polynomials can rapidly become unwieldy and of little practical utility.

The severe difficulties that can arise in spaces of many dimensions is sometimes called the **curse of dimensionality** (Bellman, 1961).

It is not limited to polynomial regression but is in fact quite general.

Consider the use of linear models for solving classification problems.

Figure 6.1 shows a plot of data from the Iris data set comprising 50 observations taken from each of three species of iris flowers.

Each observation has four variables representing measurements of the sepal length, sepal width, petal

length, and petal width.

For this illustration, we consider only the sepal length and sepal width variables.

Given these 150 observations as training data, our goal is to classify a new test point, such as the one denoted by the cross in Figure 6.1, by assigning it to one of the three species.

We observe that the cross is close to several red points, and so we might suppose that it belongs to the red class.

However, there are also some green points nearby, so we might think that it could instead belong to the green class.

It seems less likely that it belongs to the blue class.

The intuition here is that the identity of the cross should be determined more strongly by nearby points from the training set and less strongly by more distant points, and this intuition turns out to be reasonable.

One very simple way of converting this intuition into a learning algorithm would be to divide the input space into regular cells, as indicated in Figure 6.2.

When we are given a test point and we wish to predict its class, we first decide which cell it belongs to, and then we find all the training data points that fall in the same cell.

The identity of the test point is predicted to be the same as the class having the largest number of training points in the same cell as the test point (with ties being broken at random).

We can view this as a basis function model in which there is a basis function $\phi_i(\mathbf{x})$ for each grid cell, which simply returns zero if $\mathbf{x}$ lies outside the grid cell, and otherwise returns the majority class of the training data points that fall inside the cell.

The output of the model is then given by the sum of the outputs of all the basis functions.

There are numerous problems with this naive approach, but one of the most severe becomes apparent when we consider its extension to problems having larger numbers of input variables, corresponding to input spaces of higher dimensionality.

The origin of the problem is illustrated in Figure 6.3, which shows that, if we divide a region of a space into regular cells, then the number of such cells grows exponentially with the dimensionality of the

space.

The challenge with an exponentially large number of cells is that we would need an exponentially large quantity of training data to ensure that the cells are not empty.

We have already seen in Figure 6.2 that some cells contain no training points.

Hence, a test point in such cells cannot be classified.

Clearly, we have no hope of applying such a technique in a space of more than a few variables.

The difficulties with both the polynomial regression example and the Iris data classification example arise because the basis functions were chosen independently of the problem being solved.

We will need to be more sophisticated in our choice of basis functions if we are to circumvent the curse of dimensionality.

2 6.1.2 High-dimensional spaces

First, however, we will look more closely at the properties of spaces with higher dimensionality where our geometrical intuitions, formed through a life spent in a space of three dimensions, can fail badly.

As a simple example, consider a hypersphere of radius $r = 1$ in a space of D dimensions, and ask what is the fraction of the volume of the hypersphere that lies between radius $r = 1 - \epsilon$ and $r = 1$.

We can evaluate this fraction by noting that the volume $V_D(r)$ of a hypersphere of radius r in D dimensions must scale as r^D , and so we write

$$V_D(r) = K_D r^D \tag{4}$$

where the constant K_D depends only on D .

Thus, the required fraction is given by

$$\frac{V_D(1) - V_D(1 - \epsilon)}{V_D(1)} = 1 - (1 - \epsilon)^D, \tag{5}$$

which is plotted as a function of ϵ for various values of D in Figure 6.4.

We see that, for large D , this fraction tends to 1 even for small values of ϵ .

Thus, we arrive at the remarkable result that, in spaces of high dimensionality, most of the volume of a hypersphere is concentrated in a thin shell near the surface!

As a further example of direct relevance to machine learning, consider the behaviour of a Gaussian distribution in a high-dimensional space.

If we transform from Cartesian to polar coordinates and then integrate out the directional variables, we obtain an expression for the density $p(r)$ as a function of radius r from the origin.

写出多维正态分布的密度函数，分别使用直角坐标和极坐标。

练习：求二维标准正态分布的随机变量取值的模长小于 r 的概率 $F(r)$ ，然后计算密度 $F'(r)$ 。

Thus, $p(r)\delta r$ is the probability mass inside a thin shell of thickness δr located at radius r .

This distribution is plotted, for various values of D , in Figure 6.5, and we see that for large D , the probability mass of the Gaussian is concentrated in a thin shell at a specific radius.

In this book, we make extensive use of illustrative examples involving one or two variables, because this makes it particularly easy to visualize these spaces graphically.

The reader should be warned, however, that not all intuitions developed in spaces of low dimensionality will generalize to situations involving many dimensions.

Finally, although we have talked about the curse of dimensionality, there can also be advantages to working in high-dimensional spaces.

Consider the situation shown in Figure 6.6.

We see that this data set, in which each data point consists of a pair of values (x_1, x_2) , is linearly separable, but when only the value of x_1 is observed, the classes have a strong overlap.

The classification problem is therefore much easier in the higher-dimensional space.

3 6.1.3 Data manifolds

With both the polynomial regression model and the grid-based classifier in Figure 6.2, we saw that the number of basis functions grows rapidly with dimensionality, making such methods impractical for applications involving even a few dozen variables, never mind the millions of inputs that often arise

with, say, image processing.

The problem is that the basis functions are fixed ahead of time and do not depend on the data, or indeed even on the specific problem being solved.

We need to find a way to create basis functions that are tuned to the particular application.

Although the curse of dimensionality certainly raises important issues for machine learning applications, it does not prevent us from finding effective techniques applicable to high-dimensional spaces.

One reason for this is that real data will generally be confined to a region of the data space having lower effective dimensionality.

Consider the images shown in Figure 6.7.

Each image is a point in a high-dimensional space whose dimensionality is determined by the number of pixels.

Because the objects can occur at different vertical and horizontal positions within the image and in different orientations, there are three degrees of freedom of variability between images, and a set of images will, to a first approximation, live on a three-dimensional **manifold** embedded within the high-dimensional space.

练习：写出这个三维流形的定义方程。
这个三维流形与平面的仿射变换群的轨道有什么联系？

Due to the complex relationships between the object position or orientation and the pixel intensities, this manifold will be highly nonlinear.

In fact, the number of pixels is really an artefact of the image generation process since they represent measurements of a continuous world.

Capturing the same image at a higher resolution increases the dimensionality D of the data space without changing the fact that the images still live on a three-dimensional manifold.

If we can associate localized basis functions with the data manifold, rather than with the entire high-dimensional data space, we might expect that the number of required basis functions would grow exponentially with the dimensionality of the manifold rather than with the dimensionality of the data space.

Since the manifold will typically have a much lower dimensionality than the data space, this represents a huge improvement.

Effectively, neural networks learn a set of basis functions that are adapted to data manifolds.

Moreover, for a particular application, not all directions within the manifold may be significant.

For example, if we wish to determine only the orientation, and not the position, of the object in Figure 6.7, then there is only one relevant degree of freedom on the manifold and not three.

Neural networks are also able to learn which directions on the manifold are relevant to predicting the desired outputs.

Another way to see that real data is confined to low-dimensional manifolds is to consider the task of generating random images.

In Figure 6.8 we see examples of natural images along with examples of synthetic images of the same resolution generated by sampling each of the red, green, and blue intensities at each pixel independently at random from a uniform distribution.

We see that none of the synthetic images look at all like natural images.

The reason is that these random images lack the very strong correlations between pixels that natural images exhibit.

For example, two adjacent pixels in a natural image have a much higher probability of having the same, or very similar, colour, than would two adjacent images in the random examples.

Each of the images in Figure 6.8 corresponds to a point in a high-dimensional space, yet natural images cover only a tiny fraction of this space.

图像的数字存储空间维数很高，但是自然界的图像只占据了小部分，大部分是随机图像。

4 6.1.4 Data-dependent basis functions

We have seen that simple basis functions that are chosen independently of the problem being solved can run into significant limitations, particularly in spaces of high dimensionality.

If we want to use basis functions in such situations, then one approach would be to use expert knowledge to hand-craft the basis functions in a way that is specific to each application.

For many years, this was the mainstream approach in machine learning.

Basis functions, often called **features**, would be determined by a combination of domain knowledge and trial-and-error.

However, this approach met with limited success and was superseded by data-driven approaches in which basis functions are learned from the training data.

Domain knowledge still plays a role in modern machine learning, but at a more qualitative level in designing network architectures where it can capture appropriate **inductive bias**, as we will see in later chapters.

Since data in a high-dimensional space may be confined to a low-dimensional manifold, we do not need basis functions that densely fill the whole input space, but instead we can use basis functions that are themselves associated with the data manifold.

One way to do this is to have one basis function associated with each data point in the training set, which ensures that the basis functions are automatically adapted to the underlying data manifold.

An example of such a model is that of **radial basis functions** (Broomhead and Lowe, 1988), which have the property that each basis function depends only on the radial distance (typically Euclidean) from a central vector.

这个中心向量是怎么确定的？

If the basis centres are chosen to be the input data values $\{\mathbf{x}_n\}$ then there is one basis function $\phi_n(\mathbf{x})$ for each data point, which will therefore capture the whole of the data manifold.

A typical choice for a radial basis function is

$$\phi_n(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}_n\|^2}{s^2}\right) \quad (6)$$

where s is a parameter controlling the width of the basis function.

Although it can be quick to set up such a model, a major problem with this technique is that it becomes computationally unwieldy for large data sets.

Moreover, the model needs careful regularization to avoid severe over-fitting.

A related approach, called a **support vector machine** or **SVM** (Vapnik, 1995; Schölkopf and Smola, 2002; Bishop, 2006), addresses this by again defining basis functions that are centred on each of the training data points and then selecting a subset of these automatically during training.

As a result, the effective number of basis functions in the resulting models is generally much smaller than the number of training points, although it is often still relatively large and typically increases with the size of the training set.

Support vector machines also do not produce probabilistic outputs, and they do not naturally generalize to more than two classes.

Methods such as radial basis functions and support vector machines have been superseded by deep neural networks, which are much better at exploiting very large data sets efficiently.

Moreover, as we will see later, neural networks are able to learn deep **hierarchical representations**, which are crucial to achieving high prediction accuracy in more complex applications.